FUCAPE
BUSINESS SCHOOL

Compressão de Texto e de Índices Invertidos

PLN e Sistemas de Recuperação de Informação

Prof. Dr. Wagner Perin
FUCAPE Business School

Compressão de Texto e de Índices Invertidos

Leis Empíricas de RI, Variable Byte Encoding e Códigos Bit-Level

Disciplina: PLN e Sistemas de Recuperação da Informação
Programa: Ciência de Dados para Negócios | FUCAPE
Carga Horária: 4 h/a (200 min) **Prof.:** Prof. Dr. Wagner Perin
Unidade Curricular: UNIDADE 2 — Indexação e Estruturas



INSIGHT EXECUTIVO

Objetivo Central da Aula:
Dominar as leis empíricas de Heaps e Zipf e implementar algoritmos de compressão (d-gaps, Variable Byte e Elias) capazes de reduzir em 75% o consumo de memória RAM e eliminar o gargalo de barramento (*Memory Wall*).

Resgate da Aula 04 & O Paradoxo do Crescimento

A Barreira dos Inteiros Fixos de 32 Bits em Grandes Acervos

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Dilema Arquitetural: Na Aula 04, superamos o desperdício das matrizes termo-documento ($99,9\%$ de zeros) construindo o Índice Invertido. Mas o que ocorre quando o catálogo escala para centenas de milhões de documentos armazenando DocIDs como inteiros fixos de 32 bits?

O Avanço da Aula 04 vs. A Nova Barreira de Escala:

- **Conquista da Aula 04:** Eliminamos a varredura linear exaustiva $O(|D|)$ e implementamos cruzamento de postagens em tempo linear $O(L_1 + L_2)$.
- **A Suposição Frágil:** Cada entrada de postagem foi representada com inteiros convencionais de 32 bits (4 bytes) ou 64 bits (8 bytes).
- **A Realidade de Grandes Acervos:** Em coleções com 100 milhões de documentos, termos frequentes (*"frete"*, *"promoção"*, *"brasil"*) possuem **mais de 30 milhões de entradas**.
- **O Impacto Físico:** Uma única lista invertida ultrapassa **120 MB de RAM**, saturando a capacidade de servidores corporativos de produção.

A Dor de Negócio: O "Memory Wall" e Custos de Nuvem

Saturação de Barramento e Explosão da Fatura AWS em Grandes Catálogos

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Estudo de Caso: Plataforma de comércio eletrônico com 100 milhões de produtos enfrenta explosão na fatura de nuvem e colapso de latência sob tráfego de pico.

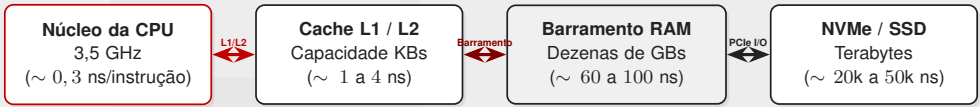
O Cenário de Crise de Infraestrutura:

- **Cluster Superdimensionado:** Servidores AWS de memória otimizada (`r5b.16xlarge` com 512 GB de RAM).
- **Fatura Mensal Fixa: US$ 14.800,00/mês** apenas para hospedar o índice bruto não-comprimido em RAM.
- **O Fenômeno do Gargalo:** Sob 4.000 buscas/s, os núcleos de CPU operavam com **80% de tempo ocioso**.
- **Diagnóstico de Engenharia:** CPUs aguardavam transferência de dados pelo barramento de memória (**Memory Wall**).



O Paradoxo do Hardware Moderno (CPU vs. Barramento)

Por que Descomprimir em RAM é Mais Rápido que Transferir Dados Brutos



INSIGHT EXECUTIVO

A Regra de Ouro da Arquitetura de Sistemas Modernos:
A CPU processa dados centenas de vezes mais rápido do que a RAM é capaz de entregá-los. Portanto, **transferir fluxos compactos pelo barramento e gastar ciclos de CPU para descomprimi-los no cache é substancialmente MAIS RÁPIDO do que transferir dados brutos não-comprimidos!**

A Anatomia do Desperdício: O Inteiro de 32 Bits

Por que Alocar 4 Bytes para Números Pequenos Inviabiliza a Escala

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Problema Investigado: Se a grande maioria dos identificadores e intervalos entre documentos é pequena, por que gastar 32 bits (4 bytes) para guardar números que cabem em 3 ou 4 bits?

Representação Binária do Inteiro 3 em 32 bits (Big-Endian):

Código Fonte

Layout de Memória (32 bits) [00000000] [00000000] [00000000] [00000011] <————————-
30 bits de ZEROS inúteis ————————> <- 2 bits úteis ->

- **Taxa Estrutural de Desperdício:** $30/32 = \mathbf{93,75\%}$ de bits redundantes em cada registro.
- **Projeção em Grande Escala (10.000.000 de postagens):**
  - Inteiros fixos de 32 bits: $10.000.000 \times 4$ bytes $= \mathbf{40}$ MB.
  - Se cada registro ocupar em média 1 byte: $10.000.000 \times 1$ byte $= \mathbf{10}$ MB ($\mathbf{75\%}$ de economia direta!).

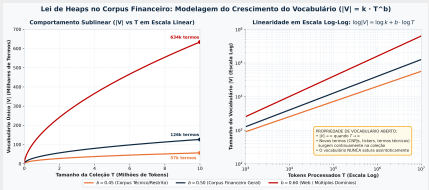
Linguística Quantitativa I — A Lei de Heaps

O Crescimento Sublinear Contínuo do Vocabulário em Línguas Naturais

Como o tamanho do vocabulário ($|V|$) se comporta ao indexar novos documentos?

$$|\mathbf{V}| = \mathbf{k} \cdot \mathbf{T}^{\mathbf{b}}$$

- T: Número total de tokens processados no corpus.
- k: Parâmetro empírico da coleção ($30 \leq k \leq 100$).
- b: Expoente sublinear, tipicamente $b \approx 0,49 \ldots 0,5$.



INSIGHT EXECUTIVO

Insight Central de Engenharia:
A curva de Heaps **nunca atinge uma assíntota horizontal**. Em línguas vivas, neologismos, nomes próprios, jargões e erros tipográficos fazem o léxico crescer indefinidamente!

Aplicação de Heaps: Dimensionamento de Memória

Projeção Prática do Impacto do Vocabulário na Memória RAM

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Aplicação Prática em Engenharia de Dados: Dimensionando o impacto do crescimento do vocabulário sobre a memória RAM do servidor de busca.

Projeção Numérica para Coleções Reais ($k = 44$, $b = 0,5$):

- **Coleção Pequena ($T = 100.000$ tokens):** $|V| = 44 \times (100.000)^{0,5} \approx \mathbf{13.914}$ termos
- **Coleção Média ($T = 1.000.000$ tokens):** $|V| = 44 \times (1.000.000)^{0,5} \approx \mathbf{44.000}$ termos
- **Coleção Corporativa ($T = 100.000.000$ tokens):** $|V| = 44 \times (100.000.000)^{0,5} \approx \mathbf{440.000}$ termos

Consequência para o Dicionário em RAM:

- Para 440.000 termos $\times$ (20B string + 16B referências em tabela hash):

$$\text{Memória Bruta} = 440.000 \times 36 \text{ bytes} \approx \mathbf{15,8} \text{ MB apenas de léxico.}$$

- **Conclusão:** Sem compressão do dicionário, o léxico se torna um entrave na memória rápida!

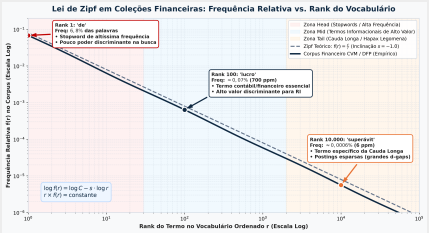
Linguística Quantitativa II — A Lei de Zipf

Distribuição de Potências e a Disparidade Extrema de Frequências

Como a frequência das palavras se distribui no corpus?

$$\mathbf{cf_i} \propto \frac{\mathbf{1}}{\mathbf{i}} \implies \mathbf{cf_i} = \frac{\mathbf{c}}{\mathbf{i}}$$

- cf_i: Frequência da palavra no i-ésimo rank.
- c: Constante normalizadora da coleção.



A Assimetria Extrema da Linguagem:

- O termo mais comum ($i = 1$) ocorre o dobro do 2º colocado ($i = 2$).
- O termo $i = 1$ ocorre **dez vezes mais** que o 10º colocado ($i = 10$).
- Poucas palavras respondem por metade de todo o texto!

Aplicação de Zipf: A Distribuição dos Gaps de Postagens

Por que as Palavras Mais Frequentes Produzem Intervalos Minúsculos

Comportamento em Corpus Financeiro Corporativo:

Rank (i)	Termo Típico	Frequência Relativa (cf_i)	Distância Média entre Ocorrências	Comportamento da Lista
1	*"de"*	$\sim 10,0\%$ dos tokens	$\approx \mathbf{10}$ tokens (Gaps $\leq \mathbf{2}$)	Lista gigante, ultradensa
10	*"empresa"*	$\sim 1,0\%$ dos tokens	$\approx \mathbf{100}$ tokens (Gaps $\approx \mathbf{5}$)	Lista média, gaps pequenos
1.000	*"debêntures"*	$\sim 0,01\%$ dos tokens	$\approx \mathbf{10.000}$ tokens (Gaps $\approx \mathbf{500}$)	Curta, gaps intermediários
50.000	*"superávit"*	$\sim 0,0002\%$ dos tokens	$\approx \mathbf{500.000}$ tokens (Gaps $> \mathbf{10.000}$)	Rara, saltos enormes

INSIGHT EXECUTIVO

Princípio Orientador da Compressão de RI:
As palavras mais volumosas da coleção produzem **listas com gaps minúsculos**. Se codificarmos números pequenos de forma ultracompacta, compactaremos a esmagadora maioria do índice de busca!

Compressão de Dicionário I — String Contínua & Blocking

Eliminando o Overhead de Ponteiros Fixos por Termo no Léxico

Abordagem Clássica: Ponteiro Fixo por Termo

- Ponteiro 1 (8B) → `"automata"` (8B)
- Ponteiro 2 (8B) → `"automate"` (8B)
- Ponteiro 3 (8B) → `"automatic"` (9B)
- Ponteiro 4 (8B) → `"automation"` (10B)

Desperdício: 32 bytes em referências para 35B de texto!

Dictionary-as-a-String + Blocking ($k = 4$)

- **1 Ponteiro de Bloco (8B)** aponta para array contíguo:

`"8automata8automate9automatic10automation..."`

- Termos internos usam byte de tamanho prefixado.

Economia: Elimina 75% dos ponteiros de navegação!

Otimizações do Léxico em RAM:

- **Dictionary-as-a-String:** Concatena termos em um array contíguo, eliminando o cabeçalho de string por objeto.
- **Agrupamento em Blocos (*Blocking*, $k = 4$):** Guarda ponteiros apenas para o primeiro termo de cada bloco de 4 palavras; preserva busca binária eficiente por salto direto.

Compressão de Dicionário II — Front Coding

Fatoração de Prefixos Comuns em Vocabulários Ordenados Alfabeticamente

Bloco Canônico $k = 4$ (Manning et al., 5.1.3):

- Termos: `automata` (8B), `automate` (8B), `automatic` (9B), `automation` (10B).
- Prefixo comum compartilhado: `"automat"` (7 chars).

Fita Contínua vs. Front Coding

Não-Fatorado (39 bytes):

```
8automata8automate9automatic10automation
```

Com Front Coding Canônico (19 caracteres):

```
8automat*a1e2ic3ion
```

Ganhos de Engenharia:
De **39B para 19 chars** ($> 51\%$ na fita e $> 60\%$ global de RAM). Preserva busca binária direta por bloco!

Anatomia de `8automat*a1e2ic3ion`:

- `8` + `automat*a`: Raiz (7) + sufixo `a` $\implies$ **automata**
- `1e`: Sufixo `e` (1 char) $\implies$ *automat* + `e` = **automate**
- `2ic`: Sufixo `ic` (2 chars) $\implies$ *automat* + `ic` = **automatic**
- `3ion`: Sufixo `ion` (3 chars) $\implies$ *automat* + `ion` = **automation**

Contagem Rigorosa de Caracteres:

$\mathbf{1}_{(8)} + \mathbf{7}_{(\text{automat})} + \mathbf{1}_{(*)} + \mathbf{1}_{(a)} + \mathbf{2}_{(1e)} + \mathbf{3}_{(2ic)} + \mathbf{4}_{(3ion)} = \mathbf{19}$ chars

(Não-fatorado: 35 texto $+ 4$ bytes tam. $= \mathbf{39}$ bytes)

Checkpoint Cognitivo 1 — Síntese do Dicionário

Momento de Pausa & Reflexão Ativa com a Turma

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO – Perguntas Disparadoras para Discussão

1. **Ponto de Checagem 1:** A Lei de Heaps demonstra que o vocabulário $|V|$ nunca satura. Se um engenheiro de dados alocar uma tabela estática de tamanho fixo para o léxico em produção, o que acontecerá ao longo do tempo?
2. **Ponto de Checagem 2:** O *Front Coding* reduz significativamente o tamanho dos termos em memória. Por que essa técnica é aplicada **apenas** dentro de blocos pequenos ($k = 4$ ou $k = 8$) e não no dicionário inteiro de uma só vez?

INSIGHT EXECUTIVO

Transição Didática Estratégica:
O dicionário representa apenas cerca de $\mathbf{10}\%$ do espaço total de um índice de busca. Os outros $\mathbf{90}\%$ residem nas **Listas de Postagens**. Veremos agora como comprimir as postagens!

O Mecanismo das Postagens — Transformação em d-gaps

Convertendo Identificadores Gigantescos em Pequenos Deltas Numéricos

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Problema: Identificadores tornam-se gigantescos conforme o catálogo cresce (ex: DocID 100.020). Como transformá-los em números pequenos para caberem em 1 ou 2 bytes?

Monotonicidade Estrita: Postagens ordenadas crescentemente ($\text{DocID}_i > \text{DocID}_{i-1}$).

Definição e Demonstração Passo a Passo:

- $d\text{-gap}_1 = \text{DocID}_1 \quad | \quad d\text{-gap}_i = \text{DocID}_i - \text{DocID}_{i-1} \quad (i > 1)$
- DocIDs Absolutos: $[100.000, 100.004, 100.005, 100.012, 100.020]$
- $\mathbf{d}$-gaps: $[\mathbf{100.000}, \mathbf{4}, \mathbf{1}, \mathbf{7}, \mathbf{8}]$

INSIGHT EXECUTIVO

Resultado Imediato: Quatro dos cinco valores agora são números de um único dígito, consumindo frações de byte!

O Algoritmo Variable Byte (VB Code) — Arquitetura

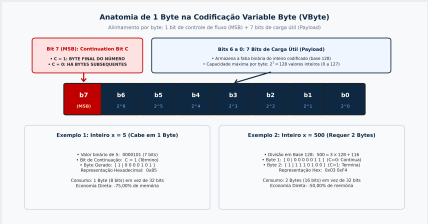
O Padrão da Indústria Adotado por Apache Lucene e Elasticsearch

Estrutura Física de Cada Byte (8 bits):

- **Bit 7 (MSB) — Bit de Continuação (C):**
  - $\mathbf{C = 1}$: Indica que este é o **último byte** do número (terminador).
  - $\mathbf{C = 0}$: Indica que o número **continua** no próximo byte.
- **Bits 6 a 0 — 7 bits de Payload Útil:** Valores de 0 a $2^7 - 1 = \mathbf{127}$ por byte.

Capacidades Operacionais de Escala:

- Valores $1 \ldots 127$: **1 único byte** gravado.
- Valores $128 \ldots 16.383$: **2 bytes** gravados.
- Alinhamento a byte permite decodificação ultrarrápida pela CPU!



Variable Byte Passo a Passo — O Caso Simples (1 Byte)

Mecânica de Codificação para Inteiros Menores que 128 ($\Delta \leq 127$)

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Mecânica Mastigada: Como codificar números inteiros menores ou iguais a 127 em exatamente 1 byte.

Exemplo Guiado: Codificando $\Delta = 5$

- **Passo 1 (Representação binária em 7 bits):** $5_{10} = 0000101_2$.
- **Passo 2 (Verificação do término):** Como $5 < 128$, este é o único e último byte necessário.
- **Passo 3 (Inserção do Continuation Bit $C = 1$):** Ativamos o MSB: Byte Final $= \mathbf{1}0000101_2 = \mathbf{10000101_2}$.
- **Passo 4 (Conversão Hexadecimal):** $\mathbf{10000101_2 = 0x85} \implies \mathbf{1}$ único byte gravado em disco/RAM!

INSIGHT EXECUTIVO

Economia Imediata:
O número 5 consumia 4 bytes (32 bits) no formato não-comprimido; agora ocupa apenas 1 byte (**75% de economia direta** de memória).

Variable Byte Passo a Passo — O Caso Composto

Mecânica de Codificação para Inteiros que Ultrapassam 127 ($\Delta \geq 128$)

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Mecânica Mastigada: Como decompor e codificar números inteiros maiores em múltiplos bytes por divisão sucessiva em base 128.

Exemplo Guiado: Codificando $\Delta = 135$ ($135 \div 128 = 1$ quociente, $135 \pmod{128} = 7$ resto)

- **Byte 1 (Mais Significativo):** Carga útil $1 = 0000001_2$. Continua ($C = 0$) $\implies$ Byte 1 $= \mathbf{0}0000001_2 = \mathbf{0x01}$.
- **Byte 2 (Terminador):** Carga útil $7 = 0000111_2$. É o último ($C = 1$) $\implies$ Byte 2 $= \mathbf{1}0000111_2 = \mathbf{0x87}$.

INSIGHT EXECUTIVO

Fluxo de Bytes Final: $[\mathbf{0x01}, \mathbf{0x87}] \implies \mathbf{2}$ bytes (em vez de 4 bytes originais)!

Trace de Decodificação do Variable Byte (Streaming)

Varredura Linear em Passagem Única $O(N)$ sem Perda de Fronteiras

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Como o leitor do motor de busca decodifica um fluxo de bytes contínuo sem perder a fronteira entre os números?

Algoritmo de Varredura em Passada Única ($O(N)$):

1. Inicializa o acumulador: `n = 0`.
2. Para cada byte lido no fluxo:
   - Se byte < 128 ($C = 0$): Acumula $\texttt{n} = 128 \times \texttt{n} + \text{byte}$.
   - Se byte ≥ 128 ($C = 1$): Finaliza $\texttt{n} = 128 \times \texttt{n} + (\text{byte} - 128)$, emite o número e reseta $\texttt{n} = 0$.

Trace Passo a Passo com o Fluxo de Entrada: `[0x85, 0x01, 0x87]`

- **Byte 1 (`0x85` = 133):** $133 \geq 128 \implies C = 1$. Número $= 133 - 128 = \mathbf{5}$. Reseta acumulador.
- **Byte 2 (`0x01` = 1):** $1 < 128 \implies C = 0$. Acumulador $= 128 \times 0 + 1 = 1$. Continua leitura.
- **Byte 3 (`0x87` = 135):** $135 \geq 128 \implies C = 1$. Número $= 128 \times 1 + (135 - 128) = \mathbf{135}$.

Números Recuperados com Sucesso: $[\mathbf{5}, \mathbf{135}]$

Checkpoint Cognitivo 2 — Dinâmica de Codificação

Exercício Prático Guiado de Fixação de d-gaps e Variable Byte

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO – Desafio Prático Corporativo: Postagens de Produtos

Considere a sequência real de DocIDs de uma lista de postagens:

$$\text{DocIDs: } [450, 458, 470, 610, 625]$$

Passo a Passo Guiado de Resolução:

1. **Passo 1 (Cálculo dos d-gaps diferenciais):**
   $\Delta_1 = 450,\ \Delta_2 = 8,\ \Delta_3 = 12,\ \Delta_4 = 140,\ \Delta_5 = 15 \implies \mathbf{d}\text{-gaps: } [\mathbf{450}, \mathbf{8}, \mathbf{12}, \mathbf{140}, \mathbf{15}]$
2. **Passo 2 (Mapeamento de Bytes em Variable Byte):**
   $450 \implies 2$ bytes $(450 \geq 128)$; $\quad 8, 12, 15 \implies 1$ byte cada; $\quad 140 \implies 2$ bytes
   $\implies$ Total de Bytes em VB Code: $2 + 1 + 1 + 2 + 1 = \mathbf{7}$ bytes.
3. **Passo 3 (Comparativo com Inteiros Fixos de 32 bits):**
   Não-comprimido: 5×4 bytes $= \mathbf{20}$ bytes $\implies$ Economia: $(1 - 7/20) \times 100 = \mathbf{65,0\%}$!

Códigos Bit-Level — Fundamentos do Elias-Gamma (γ)

Codificação em Nível de Bit para Listas Ultradensas

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

E se precisarmos de compressão ainda maior para termos extremamente frequentes, onde quase todos os gaps são iguais a 1 ou 2?

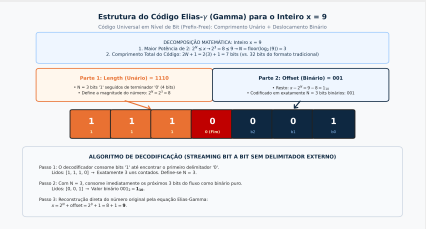
A Provocação do Alinhamento:

- No Variable Byte, mesmo o gap 1 consome 8 bits (1 byte inteiro).
- É possível codificar números usando **apenas os bits estritamente necessários**?

A Mecânica do Código Elias-Gamma (γ):

Codifica um inteiro $x \geq 1$ em duas partes contíguas:

1. **Comprimento (*Length*):** $\lfloor \log_2 x \rfloor$ em formato unário (sequência de $\lfloor \log_2 x \rfloor$ bits '1' finalizada por '0').
2. **Resto (*Offset*):** Residual $x - 2^{\lfloor \log_2 x \rfloor}$ em binário puro com $\lfloor \log_2 x \rfloor$ bits.



Elias-Gamma — Trace Passo a Passo de Codificação

Decomposição Exata entre Comprimento Unário e Offset Residual

Número (x)	$\lfloor \log_2 x \rfloor$	Parte Unária (Length)	Offset ($x - 2^{\lfloor \log_2 x \rfloor}$)	Código Elias-γ	Total de Bits
1	0	0	(vazio)	0	**1 bit**
2	1	10	$2 - 2^1 = 0 \rightarrow 0_2$	100	**3 bits**
3	1	10	$3 - 2^1 = 1 \rightarrow 1_2$	101	**3 bits**
4	2	110	$4 - 2^2 = 0 \rightarrow 00_2$	11000	**5 bits**
9	3	1110	$9 - 2^3 = 1 \rightarrow 001_2$	1110001	**7 bits**

Trace Detalhado para o Número $x = 9$:

- $\lfloor \log_2 9 \rfloor = 3 \implies$ Cadeia unária de comprimento (3 uns e um zero): 1110.
- Offset residual: $9 - 2^3 = 9 - 8 = 1$, expresso em 3 bits binários: 001.
- Código Elias-γ Unificado: $\mathbf{1110 + 001 = 1110001_2}$ (**7 bits**, menor que 1 byte!).

Elias-Delta e o Trade-off Bit-Level vs. Byte-Aligned

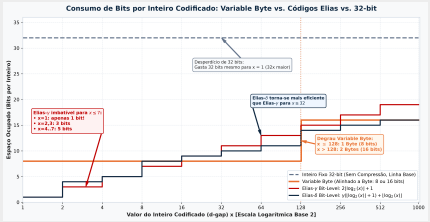
Densidade Teórica de Bits vs. Throughput de Descompressão da CPU

Limitação de Elias-γ para Gaps Maiores: Consome $2\lfloor \log_2 x \rfloor + 1$ bits. Para números altos, a cadeia unária cresce rapidamente.

- **Elias-Delta (δ):** Codifica o comprimento usando Elias-γ, otimizando números grandes.

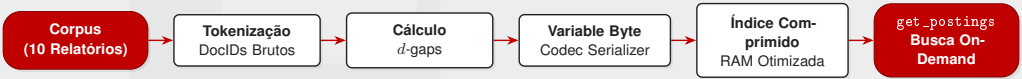
O Grande Trade-off de Engenharia:

Critério de Arquitetura	Elias-γ / δ	Variable Byte (VB)
Densidade Físico-Disco	Máxima (nível de bits)	Alta (70%–80%)
Alinhamento de Memória	Desalinhado (shifts)	**Alinhado a Byte**
Custo de Descompressão	Alto (operações bitwise)	**Baixo (CPU / SIMD)**
Cenário Recomendado	Armazenamento Frio	**Motores Online (Lucene)**



Laboratório Prático — Arquitetura da Solução em Python

Implementação do Motor de Compressão de Postagens em 4 Módulos Claros



Roteiro dos 4 Blocos de Implementação Modular:

- **Bloco 1:** `encode_number` — O conversor canônico de inteiros para bytes em formato Variable Byte.
- **Bloco 2:** `decode_stream` — O decodificador em fluxo contínuo em passada linear $O(N)$.
- **Bloco 3:** `build_compressed_index` — Geração dos d-gaps relativos e serialização compacta.
- **Bloco 4:** `get_postings` & Benchmark — Reconstrução cumulativa e validação de economia.

Laboratório — Bloco 1: O Codificador Variable Byte

Micro-Etapa 1: Implementação Canônica do Algoritmo de Codificação

```python
class VariableByteCodec:
    """Codec canonico Variable Byte (VB Code) padrao Manning et al."""
    @staticmethod
    def encode_number(n: int) -> bytes:
        """Codifica um numero inteiro usando 7 bits de dados e 1 de continuacao."""
        bytes_list = []
        while True:
            # Extrai os 7 bits menos significativos (resto por 128)
            bytes_list.insert(0, n % 128)
            if n < 128:
                break
            n //= 128
        # Liga o bit de continuacao C=1 no ultimo byte (terminador)
        bytes_list[-1] += 128
        return bytes(bytes_list)
```

Pontos Chave do Código:

- `n % 128`: Isola a carga útil de 7 bits em base 128.
- `bytes_list[-1] += 128`: Define o bit mais significativo ($C = 1$) no byte terminador.
- `bytes(bytes_list)`: Retorna a sequência nativa imutável de bytes do Python.

Laboratório — Bloco 2: O Decodificador em Streaming

Micro-Etapa 2: Leitura Linear do Fluxo de Bytes em Passagem Única $O(N)$

```python
class VariableByteCodec:
    # ... encode_number ...
    @staticmethod
    def decode_stream(bytestream: bytes) -> list[int]:
        """Decodifica um fluxo continuo de bytes em lista de inteiros originais."""
        numbers = []
        n = 0
        for byte in bytestream:
            if byte < 128:
                # C = 0: O numero continua no byte subsequente
                n = 128 * n + byte
            else:
                # C = 1: Ultimo byte encontrado! Finaliza o numero atual
                n = 128 * n + (byte - 128)
                numbers.append(n)
                n = 0
        return numbers
```

Pontos Chave do Código:

- `if byte < 128`: O MSB é zero ($C = 0$); acumula a parcela ponderada por 128.
- `else`: O MSB é um ($C = 1$); remove o marcador (`byte - 128`), salva o número e zera o registrador.

Laboratório — Bloco 3: Construção do Índice com d-gaps

Micro-Etapa 3: Integrando a Ordenação, os Deltas e o Codec ao Índice

```
class CompressedInvertedIndex:
    def __init__(self):
        self.dictionary: dict[str, dict] = {} # termo -> {'df': int, 'postings_bytes': bytes}
        self.raw_postings_uncompressed: dict[str, list[int]] = {}

    def build_compressed_index(self):
        """Converte as postagens brutas em d-gaps e codifica com Variable Byte."""
        for term, doc_ids in self.raw_postings_uncompressed.items():
            sorted_ids = sorted(doc_ids)
            # 1. Calculo dos d-gaps diferenciais
            d_gaps = [sorted_ids[0]] + [
                sorted_ids[i] - sorted_ids[i - 1] for i in range(1, len(sorted_ids))
            ]
            # 2. Codificacao compacta em fluxo de bytes
            compressed_bytes = b"".join(VariableByteCodec.encode_number(g) for g in d_gaps)
            self.dictionary[term] = {'df': len(sorted_ids), 'postings_bytes': compressed_bytes}
```

Vantagem Arquitetural: A lista invertida é guardada em memória não como objetos de inteiros Python (28 bytes cada), mas como array compacto e contíguo de bytes puros!

Laboratório — Bloco 4: Descompressão sob Demanda

Micro-Etapa 4: Reconstituindo DocIDs por Soma Cumulativa em Tempo de Busca

```python
class CompressedInvertedIndex:
    # ... build_compressed_index ...
    def get_postings(self, term: str) -> list[int]:
        """Recupera e descomprime a lista de DocIDs originais em tempo de consulta."""
        if term not in self.dictionary:
            return []
        compressed_bytes = self.dictionary[term]['postings_bytes']
        # Decodifica a sequencia de d-gaps do fluxo binario
        d_gaps = VariableByteCodec.decode_stream(compressed_bytes)
        # Reconstroi os identificadores absolutos somando os deltas cumulativos
        doc_ids, current_id = [], 0
        for gap in d_gaps:
            current_id += gap
            doc_ids.append(current_id)
        return doc_ids
```

Garantia Algorítmica: Como cada $gap_i \geq 1$, a reconstrução cumulativa é estritamente crescente e com recuperação determinística de 100% sem perdas (*lossless*).

Validação Experimental & Benchmark no Corpus

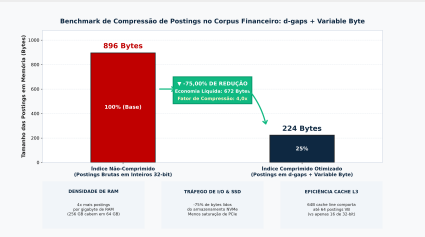
Redução Físico-Digital de 75% Mantendo Integridade Absoluta

Saída Real do Terminal de Testes:

Código Fonte

Benchmark de Compressão de Postings Nao-Comprimido
(32 bits / 4B): 896 bytes
Comprimido (d-gaps + VB): 224 bytes
Taxa de Economia em Disco/RAM: 75,00%

Postings para 'dividendos': [1, 2, 4]
-> DocID 1: Relatorio Financeiro 01
-> DocID 2: Demonstrativo de Dividendos
-> DocID 4: Remuneracao aos Acionistas

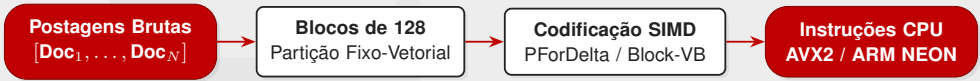


INSIGHT EXECUTIVO

Validação Experimental:
Exatamente **75,00% de redução no tamanho físico** do índice, mantendo integridade perfeita dos resultados de busca.

A Engenharia em Produção: O Apache Lucene e SIMD

Como Motores Industriais Escalam a Descompressão via Hardware Vetorial



Como Sistemas Industriais Escalam essa Lógica:

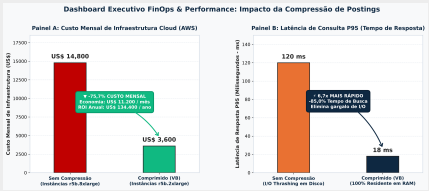
- **Blocos Fixos de 128 Números:** O Lucene particiona listas em blocos fixos, permitindo que instruções vetoriais da CPU (**SIMD**) descomprimam blocos inteiros simultaneamente em 1 a 2 ciclos de clock!
- **Índice no OS Page Cache:** A redução de 75% permite que o índice caiba no cache de disco do sistema operacional, eliminando totalmente leituras físicas de SSD/disco!

FinOps Executivo — Análise de Custos e ROI em Nuvem

O Impacto da Eficiência Algorítmica na Redução de Faturas AWS

Impacto Financeiro na Nuvem (Base de 100M SKUs):

Dimensão	Sem Comp.	Com VB	Impacto / ROI
Volume de RAM	180 GB	45 GB	**-75% de pegada**
Instância AWS	`r5b.16xlarge`	`m5.4xlarge`	Downsize 4 degraus
Custo Mensal	US$ 14.800	**US$ 3.600**	**-US$ 11.200/mês**
Custo Anual	US$ 177.600	**US$ 43.200**	**US$ 134.400**
Latência Média	120 ms (I/O)	18 **ms (Cache)**	**Aceleração 6,6x**



INSIGHT EXECUTIVO

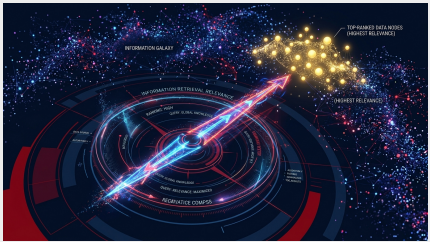
Posicionamento Executivo:
O design algorítmico gera impacto direto no balanço corporativo: a compressão viabilizou um **ROI superior a 75%** e quadruplicou a vazão do e-commerce.

Síntese da Aprendizagem & Teaser para a Aula 06

Da Compressão de Postagens ao Processamento Rankeado Top-K

O Que Dominamos na Aula 05:

1. Leis de **Heaps** (vocabulário aberto) e **Zipf** (assimetria).
2. Compressão do Dicionário (*String*, *Blocking*, *Front Coding*).
3. **Variable Byte Encoding**, d-gaps e códigos bit-level (**Elias**).
4. Alívio do **Memory Wall** e valor estratégico de **FinOps**.



INSIGHT EXECUTIVO

O Gancho Provocativo para a Aula 06:
Comprimimos o índice em 75%. Mas para o termo *"smartphone"*, a lista contém 20M documentos. Como os motores selecionam os **Top-10 produtos mais relevantes em menos de 10ms** sem pontuar todos?
$\Longrightarrow$ **Aula 06: Processamento de Consultas Lógicas e Rankeadas: Algoritmo WAND e Champion Lists!**